

Dynamo and BigTable – Review and Comparison

Grisha Weintraub

Dept. of Mathematics and Computer Science
The Open University
Raanana, Israel

Abstract — *For a long time, relational database management systems have been the only solution for persistent data storage. However, with the phenomenal growth of data, this conventional way of storing has become problematic. To manage the exponentially growing data volumes, the largest information technology companies, such as Google and Amazon, have developed alternative solutions that store data in what have come to be known as NoSQL databases. Some of the NoSQL features are flexible schema, horizontal scaling and relaxed consistency. NoSQL databases store and replicate data in distributed systems, thereby achieving scalability and high availability. In this paper two NoSQL databases are reviewed and compared: Amazon's Dynamo and Google's BigTable.*

Index Terms— Cloud Storage, Database Management Systems, Distributed Databases, NoSQL.

1. INTRODUCTION

In the recent decade, new kind of databases, often referred to as NoSQL [1], had become popular. The main characteristic of these databases is a fact that they are not relational. Rather than store data in heavily structured tables, NoSQL systems prefer simpler data schema such as key-value pairs or collections of documents. They store and replicate data in distributed systems, commonly across datacenters, by which means achieving scalability and high availability. Most of today's NoSQL databases are open source; nonetheless, many of them have been inspired by two closed source systems – Amazon's Dynamo [2] and Google's BigTable [3].

Even though open source NoSQL databases were reviewed and compared in the past [4], no comparison of Dynamo and BigTable is available. In this paper the most important features and techniques of these systems are described and a wide comparison of the two is provided.

The paper is structured as follows: section 2 provides an overview of Dynamo and BigTable, section 3 compares them, section 4 presents their open source derivatives and section 5 concludes the paper.

2. OVERVIEW

A. Dynamo

In 2007, Amazon published a paper [2] that introduced Dynamo – a highly available and scalable distributed storage system. The aim of Dynamo is to provide an underlying storage system inside Amazon's platform, which could be used

by Amazon's internal services as an "always available" data store. According to the authors of the paper, Amazon's customers "should be able to view and add items to their shopping cart even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados". To achieve such a high level of availability, Dynamo sacrifices consistency under certain scenarios.

The key Dynamo's features are:

1) *Key-value data model*: Data are stored as objects which are identified by unique keys. Only two operations on data are supported – get/put value with the specified key. This simple data model was chosen after the observation that a significant portion of Amazon's services does not need a more complicated schema.

2) *Eventual consistency*: Dynamo's main goal is to be highly available, therefore, according to the CAP theorem [5], it cannot be fully consistent. Instead, it provides an eventual consistency, that is, all updates reach all replicas eventually.

3) *Symmetry and decentralization*: Dynamo is a completely decentralized system where every node has the same set of responsibilities as its peers. Thus, there is no single point of failure and only minimal manual administration is needed.

B. BigTable

Just like Amazon built Dynamo for their internal use, Google developed BigTable – a distributed storage system for managing structured data. It is designed to scale to a very large size and to satisfy the needs of most Google applications. Unlike Dynamo's, BigTable's designers decided that key-value pair model is too limiting and chose a more sophisticated and flexible data model – multidimensional sorted map.

BigTable uses Google File System (GFS) [6] as a storage platform. GFS is a scalable and fault tolerant distributed file system, which works with huge files (by traditional standards). To achieve reliability and availability, files are divided into chunks, and each chunk is replicated across multiple machines.

The main features of BigTable are as follows:

1) *Original data model*: Bigtable is a sparse, distributed, persistent multidimensional sorted map. The map is indexed by a row key, column key, and a timestamp, and ordered by a row key. Client applications can access data either by row key or by range of row keys.

2) *Strong Consistency*: BigTable stores data in GFS as immutable files (*SSTables*) and does not replicate data in

BigTable level (as opposed to the GFS), therefore it can be considered as strongly consistent.

3) *Single-master*: BigTable, as well as GFS, is designed with a centralized approach – a single master node maintains all system metadata. This architecture significantly simplifies system's design.

3. COMPARISON

As shown above, Dynamo and BigTable are vastly different systems. This section compares them by a variety of criteria. The results of the comparison are summarized in Table II.

A. Data model

NoSQL databases are usually classified into 3 groups, according to their data model [1, 4]:

1) *Key-value stores*: As the name implies, in these systems data are stored as (key,value) pairs, such that key is a unique identifier and a value is an arbitrary entry. Values are completely opaque to the system, therefore all operations on data are based only on keys.

2) *Document-based stores*: Document stores resemble key-value stores, with the distinction that values are stored in a document-like structure such as JSON or XML. Since a document's content is transparent to the database, data operations can be based not just on keys but also on a document's attributes.

3) *Column-oriented stores*: In this model, data are organized in tables, which consist of row keys and column keys. Column keys are grouped into sets called *column families*. Table I provides an example of a typical table in column-oriented stores:

- *User ID* – is a row key.
- *Personal Data and Financial Data* – are column families.
- *Name, Phone, Email and Card* – are column keys.
- *Bob* – is a value located in a *cell* ("145778","Personal Data: Name")

TABLE I. EXAMPLE OF USERS TABLE IN COLUMN-ORIENTED STORE

User ID	Personal Data		Financial Data
145778	Phone = "781455"	Name = "Bob"	Card = "9875"
188548	Email = "john@g.com"	Name = "John"	Card = "6652"

Each cell can contain multiple versions of data which are indexed by timestamp. For example, cell ("145778","Personal Data: Name") may contain value "Bob" at timestamp t and value "Robert" at timestamp t' .

An important feature of column-oriented stores is that their rows are *sparse*, which means that different rows may contain different columns and even a different number of columns. For example, row 145778 in the table above contains column "Phone", while row 188548 contains column "Email".

To conclude, Dynamo belongs to the first group, while BigTable to the third.

B. API

The APIs of Dynamo and BigTable directly reflect their data model.

Dynamo exposes only two operations:

- *get* – returns an object associated with the given key.
- *put* – associates the given object with the specified key.

BigTable provides two types of supported actions – metadata operations and data operations. Metadata operations include functions for creating and deleting tables and column families. Data operations are:

- *get* – returns values from the individual rows.
- *scan* – iterates over multiple rows.
- *put* – inserts a value to the specified table's cell.
- *delete* – deletes a whole row or a specified cell inside a particular row.

C. Security

Both systems were designed for internal (i.e. trusted) use. However, while Dynamo completely ignores security related requirements, BigTable has a very adequate authorization mechanism.

In BigTable, access control rights are granted at column-family level. For example, it can be configured that three different applications can access data from Table I: the first is only permitted to view personal data, the second can view and update personal data and the third can view and update all users' data (including financial).

D. Partitioning

Dynamo and BigTable are highly scalable systems. It means that they can adjust themselves to the changing data volume by simply adding or removing nodes. This requires a mechanism to dynamically partition data over the nodes in the system, and Dynamo and BigTable do it in very different ways.

Dynamo uses a technique called *consistent hashing*, where every node in the system is assigned to one or more points on a fixed circular space called "ring". Each data item, identified by a key, is assigned to a node by hashing its key with a hash function, whose output is a point on the ring, and then walking the ring clockwise to find the first node that appears on it (*coordinator* node). The main advantage of this technique is that addition or removal of a node only affects its immediate neighbors and other nodes remain unaffected.

BigTable stores data ordered by a row key. Row ranges are called *tablets*. Each table consists of a set of tablets, and each tablet contains all data associated with a row range. Initially, each table consists of just one tablet. As a table grows, it is automatically split into multiple tablets. BigTable implementation includes single master and multiple tablet servers. The master is responsible for assigning tablets to tablet servers, whereas tablet servers are responsible for handling read and write requests to the tablets that they serve,

and splitting tablets that have grown too large. Since each table's cell belongs to a particular row, each row belongs to a particular tablet, and each tablet is assigned to exactly one tablet server at a time, it is very simple to find a node which stores the data of a specific table's cell. The only thing to do is to query a special METADATA table, which stores the location of a tablet under a row key.

E. Replication

To achieve high availability, reliability and durability, both systems replicate data on multiple hosts. However the techniques which are used by them are extremely different.

Dynamo replicates each data object at N nodes, where N is a user-defined parameter. As described in the previous section, each key K is assigned to a coordinator node. This node stores the data associated with K locally, and also replicates it at the $N-1$ healthy clockwise successor nodes in the ring.

In BigTable each table's cell belongs to a particular tablet, and each tablet is stored in GFS as a sequence of read-only files called SSTables. In GFS, SSTables are divided into fixed-size *chunks*, and these chunks are stored on *chunkservers*. System architecture of GFS resembles BigTable's – a single master, which maintains system metadata and multiple chunkservers which are responsible for the data operations. Just like tablet servers in BigTable serve set of tablets, chunkservers in GFS serve set of chunks. An important difference between these two systems is that each tablet in BigTable belongs to exactly one tablet server, whereas each chunk in GFS is replicated across multiple chunkservers (three by default, but can be configured).

F. Storage

Each node in Dynamo has a local persistence engine where data items are stored as binary objects (i.e. blob). Different Dynamo instances may use different persistence engines. Examples of the engines that can be plugged-in include MySQL and Berkeley Database (BDB). The main reason for designing a pluggable persistence component is to allow users to choose the storage engine best suited for their needs. For instance, BDB handles relatively small objects (typically tens of kilobytes) whereas MySQL can handle objects of much larger sizes. Applications choose the persistence engine based on their object size distribution.

As noted earlier, BigTable stores its data in GFS in SSTable file format. SSTable is an immutable ordered map, whose keys and values are arbitrary strings. SSTable supports "get by key" and "get by key range" requests.

G. Read/Write Implementation

To read and write replicated data, Dynamo uses a protocol similar to those used in quorum systems. This protocol has two parameters R/W – the minimal number of nodes that must participate in a successful *read/write* operation respectively. When a coordinator receives a write operation, it writes the data locally and then sends a write request to the other $N-1$ replica nodes. If at least $W-1$ nodes respond, the operation is considered successful and the coordinator informs the client. When a read operation is received, the coordinator sends a

read request to the $N-1$ nodes, and if at least $R-1$ nodes respond, it returns the result to the client. If different objects are received from different nodes, then coordinator sends to the client a list of objects with conflicting versions (see Section 3.H), rather than a single object.

Tablet servers in BigTable handle read and write requests for the tablets that they serve. Tablet data operations are performed as illustrated in Fig. 1. When a write operation arrives at a tablet server, it is written to the commit log (this log is used to recover the tablet in case of crash). After the write has been committed, its content is inserted into the *memtable* – an in-memory sorted buffer. When the memtable size reaches a certain threshold, the memtable is frozen, a new memtable is created, and the frozen memtable is converted to an SSTable and is written to GFS. From time to time all SSTables are merged to exactly one SSTable. When a read operation arrives at a tablet server, it is executed on a merged view of the sequence of SSTables and the memtable. Since the SSTables and the memtable are sorted data structures, the merge can be performed efficiently. To reduce the number of disk accesses required to read all SSTables for each read operation, BigTable uses Bloom filters. Bloom filters are kept in the main memory and allow checking whether the specific SSTable might contain any data for a specified row/column pair.

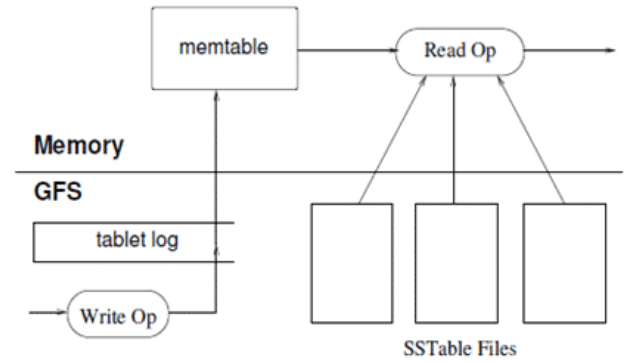


Fig. 1. Tablet data operations in BigTable (taken from [3])

H. Concurrency Control

Concurrency control deals with the issues involved with allowing multiple clients access shared objects concurrently. Not surprisingly, the implementation of concurrency control is different in each system.

Dynamo is designed as an eventually consistent system. Write operations return before all replica nodes have applied the update. As a consequence, read operations may return different versions of the same object. Dynamo's clients are designed to handle such inconsistencies. For example, if the "shopping cart" application receives different versions of the user's shopping cart, it can resolve the conflict, by updating the cart to be the merge view of all the conflicting carts. Using this reconciliation mechanism, items which were added to the cart are never lost. However, deleted items can reappear. In

order to maintain different versions of the same objects, Dynamo uses *vector clocks*. Vector clock is a list of the (node, counter) pairs, which is associated with every version of every object. By examining vector clocks of different object versions, one can determine whether the versions are in conflict and reconciliation is required or not.

BigTable guaranties that read and write operations under a single row key are atomic. As shown in Fig. 1, write operations access the tablet log and the memtable, while read operations access the memtable and the SSTables. Thus the only data structure that is accessed by both reads and writes is the memtable. To allow reads and writes of the memtable to proceed in parallel, BigTable makes each memtable row copy-on-write. This means that each time a memtable row is updated a new copy of the row is created. With this copy-on-write approach, BigTable can support concurrent read and write operations without using heavy synchronization mechanisms as locks.

I. Membership and Failure Detection

Both systems should be able to detect the addition and removal/failure of nodes as soon as possible.

Dynamo uses gossip-based protocol for membership and failure detection. Each node contacts a peer chosen at random every second and the two nodes exchange their membership data (every node maintains a persistent view of the membership).

BigTable identifies failed tablet servers by regular handshakes between the master and all tablet servers.

J. Consistency/Availability

The CAP theorem [5] states that any networked shared-data system can have at most two of three desirable properties:

- Consistency (C) - equivalent to having a single up-to-date copy of the data
- Availability (A) of that data (for reads and writes)
- Tolerance to network partitions (P)

Because of this tradeoff, it is necessary to sacrifice one of these properties. The general belief is that designers cannot sacrifice P and therefore have a difficult choice between C and A. As noted earlier and as stems from the comparison so far, Dynamo sacrifices consistency, while BigTable availability.

The tradeoff is clear. Dynamo clients do not have to wait until their updates reach all the replicas, but in return they deal with the multiple object versions on reads. Similarly, BigTable clients enjoy a consistent view of their data, but in return they must wait in the presence of system failures.

TABLE II. COMPARISON SUMMARY

Category	Dynamo	BigTable
Architecture	Decentralized	Centralized
Data model	Key-value	Column-oriented
API	get, put	get, put, scan, delete
Security	No	Access control at column-family level
Partitioning	Consistent hashing	Key range based

Category	Dynamo	BigTable
Replication	Across data centers	No replication in BigTable. Replication in GFS within a single data center.
Storage	Plug-in	SSTables in GFS
Read/Write	Quorum-like	Reads – merge of SSTables and memtable Writes – tablet log and memtable
Concurrency Control	Vector clocks with reconciliation during reads	Copy-on-write
Membership and failure detection	Gossip-based protocol	Handshakes initiated by master
CAP	AP	CP

4. DERIVATIVES

Dynamo and BigTable have inspired many modern NoSQL systems. In this section the most known are presented. The section is mainly based on the systems' websites.

A. Riak (<http://basho.com/riak>)

Riak is a Dynamo-like distributed key-value data storage system which was built by Basho. It was written in Erlang and was open-sourced in 2009. Riak uses most of the Dynamo techniques and can be considered as a direct implementation of [2]. However Riak has its own features as well. One of them is a "links" feature. Links are metadata that establish one-way relationships between objects. For example user A can be linked to the set of users S, to indicate that users in S are contacts of A. Once such links are defined, queries of type "get all contacts of user A" can be executed. Actually, not only link queries are accepted by Riak but any query in the form of a function. The processing layer that handles such queries in Riak is called Riak Pipe [7].

B. Project Voldemort (<http://www.project-voldemort.com>)

Project Voldemort is a distributed data store, built by LinkedIn, whose design and architecture have been heavily inspired by Dynamo. It was written in Java and was open-sourced in 2009. The novelty of Voldemort, compared to Dynamo, is its custom storage engine for bulk-loaded read-only data sets [8]. By using this engine, Voldemort supports bulk loading of massive data sets without affecting serving performance.

C. HBase (<http://hbase.apache.org>)

HBase is a distributed non-relational database modeled after BigTable. It is Apache top-level project and is built on top of Hadoop Distributed File System (HDFS) [9]. It is open source and is written in Java. Facebook uses HBase as a "storage infrastructure behind Facebook Messages" [10].

D. Hypertable (<http://hypertable.com>)

Similarly to HBase, Hypertable is an open source implementation of BigTable. It is written in C++. Unlike BigTable which uses only one storage layer (GFS), Hypertable is capable of running on top of any file system. To achieve this, the system has abstracted the interface to the file system

by sending all data requests through a Distributed File System (DFS) broker process.

E. Accumulo (<http://accumulo.apache.org>)

Another open-source implementation of BigTable is Accumulo. Accumulo was developed by US National Security Agency and was open sourced in 2011 as an Apache project. Just like HBase it is written in Java and uses HDFS as a storage layer. Accumulo has two notable novel features - cell-level security and iterators.

1) *Cell-Level Security*: Every key-value pair has its own security label, which is used to determine whether a given user meets the security requirements to read the value. This enables data of various security levels to be stored within the same row, and users of varying degrees of access to query the same table, while preserving data confidentiality.

2) *Iterators*: Iterators provide a modular mechanism for adding functionality to be executed by tablet servers when scanning or compacting data. This allows users to efficiently summarize, filter, and aggregate data. Cell-level security is implemented by using iterators.

In Table III main components of HBase, Hypertable and Accumulo are named along with their BigTable's analogues.

TABLE III. BIGTABLE VS. HBASE VS. HYPERTABLE VS. ACCUMULO

BigTable	HBase	Hypertable	Accumulo
GFS	HDFS	DFS	HDFS
Tablet Server	Region Server	Range Server	Tablet Server
SSTable	StoreFile	CellStore	ISAM file
Memtable	MemStore	CellCache	MemTable
Chubby	ZooKeeper	Hyperspace	ZooKeeper

F. Cassandra (<http://cassandra.apache.org>)

Cassandra is a distributed storage system, which was originally developed by Facebook [11]. It was written in Java and was open-sourced in 2008. Today it is an Apache top-level project. Cassandra adopts ideas and concepts of both Dynamo and BigTable and can be described as a BigTable-like data model running on a Dynamo-like infrastructure. Cassandra's data model is similar to BigTable's data model with only two extensions – *super column families* and a user-defined sort order. Super column family is an additional level of grouping and can be visualized as a column family within a column family. Applications can specify the sort order of columns within a column family or a super column family. Cassandra is based on decentralized (Dynamo-like) architecture. It uses techniques similar to Dynamo's for partitioning, replication, concurrency control and membership and failure detection. However the read/write operations are implemented by using both Dynamo and BigTable

approaches. Like Dynamo, Cassandra uses quorum-like protocol for reads and writes. As in BigTable, writes are first written to the commit log and then to the in-memory data structure (i.e. memtable). Storage layer of Cassandra is neither a pluggable database as in Dynamo nor a distributed file system as in BigTable; it is a local file system.

5. CONCLUSION

The goal of this paper is to provide an essential overview of the two, probably the most influencing, NoSQL databases – Dynamo and BigTable. We have reviewed and compared these systems and have discussed open source projects which have been inspired by them.

To conclude, both systems are non-relational distributed data storage systems. Dynamo uses a decentralized architecture with a key-value data model and an eventual consistency, while BigTable is built with a single master approach, a column-oriented data model and a strong consistency.

The main advantage of Dynamo over BigTable is its very high level of availability and an ability to replicate data across many data centers world-wide. The main advantage of BigTable over Dynamo is its strong consistency and ability to store structured and ordered data.

REFERENCES

- [1] Neal Leavitt: Will NoSQL Databases Live Up to Their Promise?. IEEE Computer (COMPUTER) 43(2):12-14 (2010)
- [2] G. DeCandia et al.: Dynamo: amazon's highly available key-value store. SOSP 2007:205-220
- [3] F. Chang et al.: Bigtable: A Distributed Storage System for Structured Data. OSDI 2006:205-218
- [4] Rick Cattell: Scalable SQL and NoSQL data stores. SIGMOD Record (SIGMOD) 39(4):12-27 (2010)
- [5] Seth Gilbert, Nancy A. Lynch: Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. SIGACT News (SIGACT) 33(2):51-59 (2002)
- [6] Sanjay Ghemawat, Howard Gobioff, Shun-Tak Leung: The Google file system. SOSP 2003:29-43
- [7] Bryan Fink: Distributed computation on dynamo-style distributed storage: riak pipe. Erlang Workshop 2012:43-50
- [8] Roshan Sumbaly et al.: Serving large-scale batch computed data with project Voldemort, Proceedings of the 10th USENIX conference on File and Storage Technologies, p.18-18, February 14-17, 2012, San Jose, CA
- [9] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, Robert Chansler: The Hadoop Distributed File System. MSST 2010:1-10
- [10] Amitanand S. Aiyer et al.: Storage Infrastructure Behind Facebook Messages: Using HBase at Scale. IEEE Data Eng. Bull. (DEBU) 35(2):4-13 (2012)
- [11] Avinash Lakshman, Prashant Malik: Cassandra: a decentralized structured storage system. Operating Systems Review (SIGOPS) 44(2)